Inferenza di una rete neurale nel kernel Linux

Le slide una alla volta, ognuna con la sua spiegazione

Ogni pagina ha l'immagine di una slide e sotto la spiegazione. I riquadri grigi sono **esempi con i numeri**; i riquadri gialli segnalano i **punti da correggere o da dire con cautela**; i riquadri verdi le **verifiche** fatte ricontrollando i dati.

Le parole e le unità che tornano sempre

Nodo	Il computer che riceve il pacchetto, esegue la rete neurale per scegliere da quale porta farlo uscire, e lo inoltra.
XDP, eBPF	XDP è il primo punto del kernel Linux in cui un programma può vedere un pacchetto appena arrivato; eBPF è il linguaggio di quei programmi, che il kernel controlla prima di eseguirli.
Versione (P1, P1.5, P2, P3)	Uno dei quattro modi di scrivere il programma che esegue la rete neurale: da tutto scritto nel programma (P1) a tutto letto da tabelle (P3).
Senza rete (baseline)	Il nodo che riceve e inoltra senza eseguire nessuna rete neurale: il costo fisso che ogni versione paga. «Solo ricezione» riceve e basta.
Tabella	Una zona di memoria del kernel che il programma consulta mentre gira e che si può riscrivere senza ricompilarlo (in eBPF si chiama mappa).
Salto fra programmi	Il passaggio da un programma eBPF a un altro (tail call). P3 ne fa uno per ogni strato della rete.
Compilare	Tradurre il programma in codice eseguibile. P1 si compila per ogni modello nuovo, fuori dal nodo; P2 e P3 una volta sola.
Popolazione	Le 91 reti neurali generate su cui gira ogni misura: diverse per ingressi, strati, neuroni, pesi e pesi a zero.
Mediana [min–max]	La mediana sui modelli (metà costa meno, metà di più) e, fra parentesi, il modello più leggero e il più pesante.
Legge di costo	Costo della rete ≈ parte fissa + moltiplicazioni × costo di una + strati × costo di uno, stimata su tutti i modelli e provata su modelli lasciati fuori.
Moltiplicazioni eseguite	Le moltiplicazioni che il programma fa davvero per un pacchetto (in inglese MAC). Un ingresso con molte colonne di cui una sola vale 1 ne conta una.
Capacità	Quanti pacchetti al secondo il nodo riesce a elaborare al massimo.
Costo per pacchetto	Il tempo di processore per elaborare un pacchetto: 1 diviso la capacità. 2 milioni di pacchetti al secondo = 500 ns per pacchetto.
ns, µs, ms	Nanosecondi (miliardesimi di secondo), microsecondi (1 µs = 1 000 ns), millisecondi (1 ms = 1 000 µs).

Nodo pieno	Il generatore manda più pacchetti di quanti il nodo ne regga: la coda resta piena e il core lavora il 100 % del tempo.
Coda	I posti davanti al nodo dove i pacchetti aspettano. Se è piena, chi arriva è respinto.
Collo di bottiglia	Il pezzo più lento della catena: decide quanto traffico passa.
P-core, E-core, LP E-core	I tre tipi di core del processore: veloci, efficienti (a gruppi di quattro che condividono una cache) e a basso consumo (in un'altra parte del chip, al massimo 2,5 GHz).
Ciclo di clock, IPC	Il ciclo è il battito dell'orologio del core (3,5 miliardi al secondo a 3,5 GHz). IPC = istruzioni eseguite per ciclo: dice quanto lavoro il core fa a ogni battito.
Istruzioni per pacchetto	Quante istruzioni il processore esegue per ogni pacchetto, contate dal processore stesso. Dipendono dal codice, non dal core.
Cache	Una piccola memoria veloce vicino al core. Un dato che non c'è va letto dalla memoria principale, molto più lenta.
Tre orologi	Una versione del programma legge l'orologio all'ingresso, prima di inoltrare e all'arrivo al nodo successivo: separa il tempo del programma da quello del trasporto.
Programma da solo	Il kernel esegue il programma su un pacchetto preparato, senza traffico: serve a contare letture di tabella e salti, e a misurare il cambio di modello.

Come si misura e da dove vengono i tempi

Che cosa fa il nodo. In una rete di calcolatori ogni nodo riceve pacchetti e deve decidere da quale porta farli ripartire. Qui la decisione la prende una piccola **rete neurale** che gira **dentro il kernel Linux**, nel primo punto in cui il pacchetto arriva (XDP, una funzione del kernel che lascia eseguire un programma su ogni pacchetto appena ricevuto). Il programma legge alcune informazioni (lo stato dei collegamenti, la porta d'ingresso, il TTL, cioè quanti salti di rete restano al pacchetto), le dà alla rete neurale, e inoltra il pacchetto sulla porta che la rete sceglie, oppure lo scarta.

Il banco. Un solo portatile fa tutto: il generatore di traffico, il nodo che esegue la rete neurale e il nodo successivo che riceve i pacchetti inoltrati. Sono collegati da cavi virtuali; il programma del nodo vede ogni pacchetto come lo vedrebbe da una scheda di rete vera. Ogni ruolo ha un core del processore tutto suo.

Le condizioni della macchina. Lasciato a sé stesso, il portatile cambia velocità di continuo, addormenta i core e mette altri programmi sul core che sta misurando. Prima di ogni misura il banco fissa la frequenza dei core a 3,5 GHz, impedisce che si addormentino, sposta il resto del sistema su altri core e sorveglia la macchina (frequenza, temperatura, rallentamenti per calore, alimentazione). Una misura in cui la macchina esce da queste condizioni viene marcata e non entra nelle conclusioni.

I tre tipi di core. Il processore (Intel Core Ultra 7 155H) ha core **veloci** (P-core), core **efficienti** (E-core) e due core **a basso consumo** (LP E-core), che stanno in un'altra parte del chip. Dove non è detto altro, il nodo gira su un P-core.

La popolazione di modelli. Ogni misura è ripetuta su **91 reti neurali diverse**, generate apposta. Hanno da 5 a 117 ingressi, da 1 a 6 strati nascosti, da 2 a 8 neuroni per strato, da 3 a 7 uscite, da 21 a 1 349 pesi. Sono di quattro tipi:

- il **disegno principale** (53 modelli): le combinazioni di tipo d'ingresso, numero di strati e larghezza;
- le **repliche** (20): sei forme ripetute con pesi estratti a caso in modo diverso, per vedere quanto contano i valori dei pesi;
- i **modelli con pesi a zero** (8): quattro forme con il 50 e il 90 % dei pesi uguali a zero;
- le **forme a pari pesi** (10): per ognuno dei due tipi d'ingresso, gli stessi pesi (circa 600 o circa 210–270) distribuiti su 1, 2, 3, 4 o 5 strati.

Le prove più lunghe (traffico crescente, core a basso consumo, due core, decisioni diverse, pacchetti di taglie diverse) usano 6–14 modelli, scelti dal più leggero al più pesante.

Cinque modi di misurare, ognuno per una domanda diversa:

1. **A nodo pieno.** Il generatore manda più pacchetti di quanti il nodo ne regga: il core del nodo lavora il 100 % del tempo. Si misura la **capacità** (quanti pacchetti al secondo il nodo elabora) e il **tempo per pacchetto**, che è 1 diviso la capacità. In più, il processore stesso conta istruzioni, cicli di clock e letture andate in memoria.
2. **A traffico crescente.** Il traffico sale da 0,5 a 12 milioni di pacchetti al secondo; si contano i pacchetti inviati, quelli che il programma riceve e quelli inoltrati. Dice dove si perdono i pacchetti oltre la capacità (slide 10).
3. **Con tre orologi.** Una versione del programma legge l'orologio all'ingresso, subito prima di inoltrare, e all'arrivo al nodo successivo: separa il tempo del programma da quello del trasporto, con poco traffico (slide 13).
4. **Il programma caricato.** Il kernel dice quanto è grande ogni programma, in istruzioni (slide 5, 14, 17, 19).
5. **Il programma da solo.** Il kernel esegue il programma su un pacchetto preparato, senza traffico. Serve a contare quante volte il programma legge una tabella e quante volte salta da un programma all'altro, e a misurare quanto costa installare un modello nuovo (slide 14 e 15).

Da dove vengono i tempi (P-core a 3,5 GHz, mediane sui 91 modelli):

- **ricevere il pacchetto**: circa 40 ns (un nanosecondo, ns, è un miliardesimo di secondo);
- **leggere l'intestazione e inoltrare**: circa 27 ns in più. Insieme fanno i ~67 ns del **nodo senza rete** (in inglese *baseline*): il costo fisso, uguale con qualunque modello;
- **la rete neurale**: quello che ogni versione aggiunge al nodo senza rete. 52 ns per P1, 59 per P1.5, 225 per P2, 420 per P3; dal modello più leggero al più pesante da 19 a 852 ns;
- **il trasporto** verso il nodo successivo: circa 200 ns, uguale per tutte le versioni e tutti i modelli.

> **I limiti del banco.** Il cavo virtuale non è una scheda di rete, e il generatore sta sulla stessa macchina: il costo del programma è quello vero, quello di ricezione su una scheda vera può essere diverso. Le cifre assolute valgono a 3,5 GHz. I modelli sono generati: coprono forme e tipi d'ingresso diversi, non dicono niente sull'accuratezza di un modello addestrato.

> **Quanto è pulito il dato.** Su 526 misure 2 sono fuori dalle conclusioni, perché la macchina era disturbata. Su questo portatile un breve rallentamento del processore arriva circa una volta al minuto: le curve a traffico crescente, che durano quasi tre minuti, sono tenute senza i pochi istanti colpiti (17 su 1 920).

Per chi vuole rifare le misure. Tutto lo fa `ipa/test/remeasure_popolazione.sh` (circa 4 ore e mezza); le conclusioni le calcola `ipa/test/population_report.py`, e queste slide le rigenera `docs/slides_src/popolazione.py`. I banchi sono `bench_throughput.py` (nodo pieno, tre orologi), `bench_bitrate.py` (traffico crescente), `bench_solo.py` (programma da solo), il generatore è `xdp_gen.py`, le condizioni della macchina `host_conditions.py`. La guida completa è `docs/testing.md`, §13.

SLIDE 1

Copertina

TESI · EBPF / XDP

Inferenza di una rete neurale nel kernel Linux con eBPF/XDP

Quattro modi di scrivere la stessa rete neurale dentro il kernel Linux, misurati con traffico vero su 91 reti diverse: quanto costa ogni pacchetto, perché, e che cosa cambia su un core più lento.

Ettore Cantile

ottobre 2026

La tesi esegue una piccola rete neurale **dentro il kernel Linux**, nel punto in cui il pacchetto è appena arrivato, per decidere da quale porta farlo uscire. La stessa rete è scritta in quattro modi, da completamente fissata nel programma a completamente configurabile, e la domanda è: quanto costa, per ogni pacchetto, ciascuna scelta? E perché?

La risposta non deve dipendere da un modello particolare. Per questo ogni misura è ripetuta su **91 reti neurali diverse**, e ogni conclusione è un intervallo o una legge che vale su tutte.

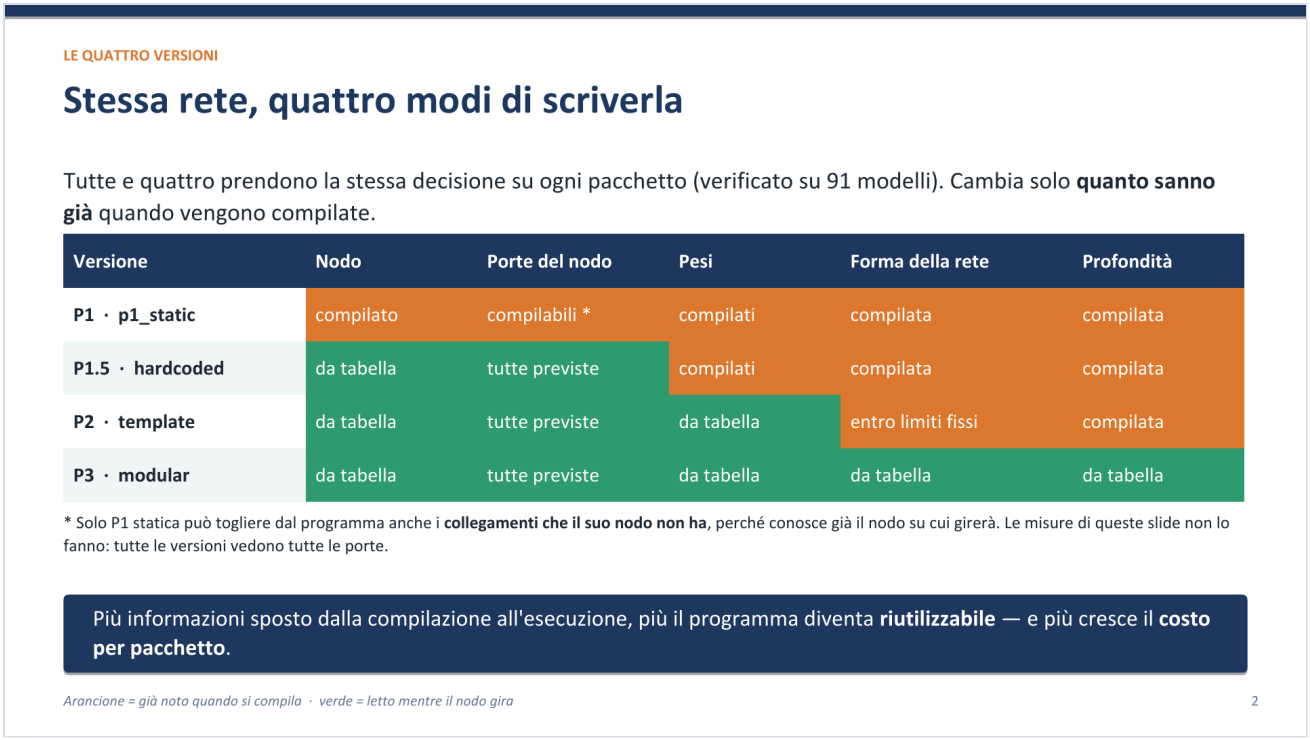
La presentazione segue cinque passi:

1. **Il nodo** (slide 4–5): quanto costa un pacchetto e quanto lavora il processore per elaborarlo.
2. **Tre tipi di core** (slide 6–8): che cosa cambia se il nodo gira su un core efficiente o a basso consumo.
3. **Sotto carico** (slide 9–11): che cosa succede quando il traffico supera la capacità, con due core, e che cosa non influisce sul costo.
4. **La forma del modello** (slide 12–13): quanto pesano strati e neuroni sul nodo intero.
5. **La sola rete neurale** (slide 14–19): il prezzo della configurabilità, il cambio di modello, e una legge che prevede il costo di un modello prima di misurarlo.

Le cifre vengono dalla campagna dell'11–12 ottobre 2026 sul portatile di laboratorio. Prima delle slide, la pagina «Come si misura e da dove vengono i tempi» descrive il banco, i modelli e i modi di misurare; le spiegazioni la danno per letta.

SLIDE 2

Stessa rete, quattro modi di scriverla



LE QUATTRO VERSIONI

Stessa rete, quattro modi di scriverla

Tutte e quattro prendono la stessa decisione su ogni pacchetto (verificato su 91 modelli). Cambia solo **quanto sanno già** quando vengono compilate.

Versione	Nodo	Porte del nodo	Pesi	Forma della rete	Profondità
P1 · p1_static	compilato	compilabili *	compilati	compilata	compilata
P1.5 · hardcoded	da tabella	tutte previste	compilati	compilata	compilata
P2 · template	da tabella	tutte previste	da tabella	entro limiti fissi	compilata
P3 · modular	da tabella	tutte previste	da tabella	da tabella	da tabella

* Solo P1 statica può togliere dal programma anche i **collegamenti che il suo nodo non ha**, perché conosce già il nodo su cui girerà. Le misure di queste slide non lo fanno: tutte le versioni vedono tutte le porte.

Più informazioni sposto dalla compilazione all'esecuzione, più il programma diventa **riutilizzabile** — e più cresce il **costo per pacchetto**.

Arancione = già noto quando si compila · verde = letto mentre il nodo gira

2

Tutte e quattro le versioni calcolano **la stessa rete neurale** e prendono **la stessa decisione** su ogni pacchetto. Lo verifica, su ogni modello, un confronto esatto con un calcolo di riferimento fatto fuori dal kernel: 200 pacchetti per modello, nessuna differenza. P2 accetta al più 1 024 pesi, quindi per P2 i modelli sono 85 invece di 91.

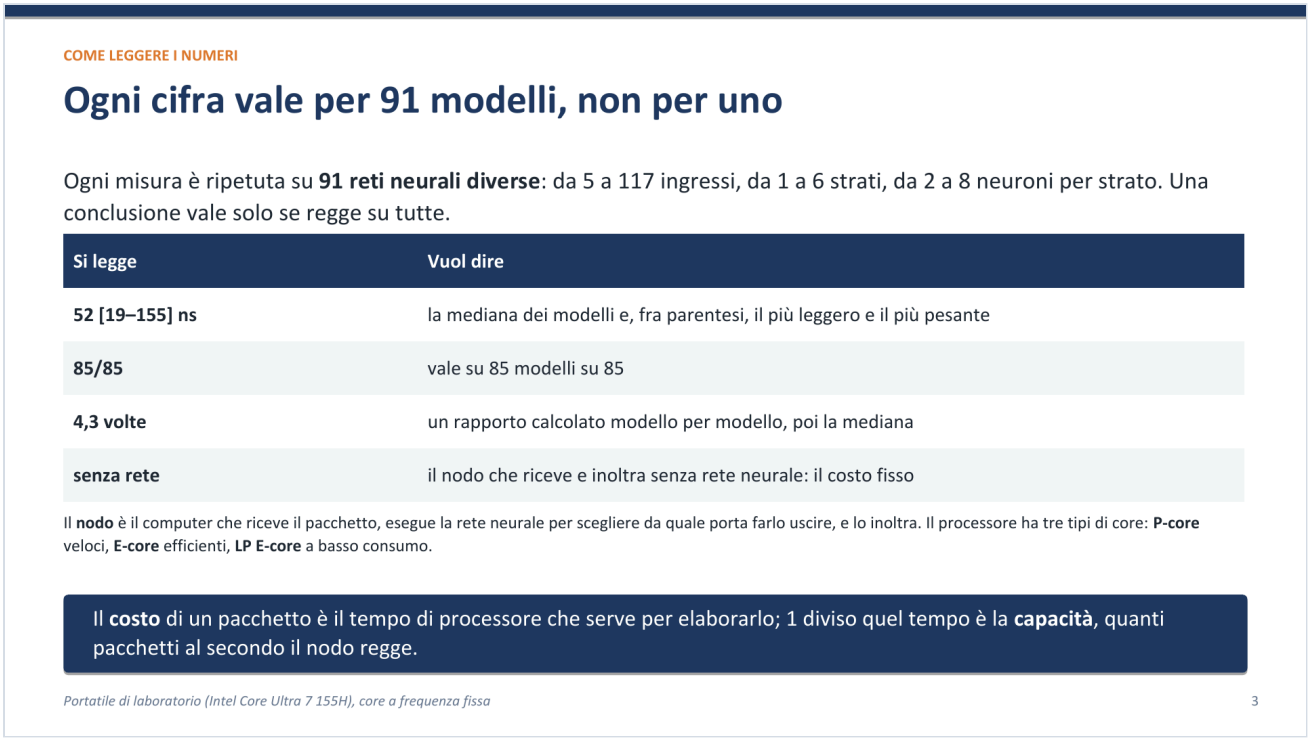
Le versioni differiscono in **che cosa è già scritto nel programma quando lo si compila** (arancione nella tabella) e **che cosa il programma legge da tabelle mentre gira** (verde). Una tabella, qui, è una zona di memoria del kernel che il programma consulta e che si può riscrivere senza ricompilare il programma.

- **P1 statica**: i pesi e l'identità del nodo sono scritti nel programma. Serve un programma diverso per ogni nodo.
- **P1.5**: i pesi sono nel programma, il nodo si legge da tabella. Un programma per ogni modello.
- **P2**: anche la forma della rete (quanti ingressi, quanti neuroni) si legge da tabella; nel programma ci sono solo i valori massimi ammessi. Un programma per molti modelli.
- **P3**: anche il numero di strati si legge da tabella, e ogni strato è un programma a parte, raggiunto con un salto. Un programma per tutto.

La regola che le slide successive misurano: più informazioni si spostano dalla compilazione all'esecuzione, più il programma diventa riutilizzabile, e più cresce il costo per pacchetto. I colori restano gli stessi in tutti i grafici: P1 rosso, P1.5 arancione, P2 verde, P3 blu.

SLIDE 3

Ogni cifra vale per 91 modelli, non per uno



COME LEGGERE I NUMERI

Ogni cifra vale per 91 modelli, non per uno

Ogni misura è ripetuta su **91 reti neurali diverse**: da 5 a 117 ingressi, da 1 a 6 strati, da 2 a 8 neuroni per strato. Una conclusione vale solo se regge su tutte.

Si legge	Vuol dire
52 [19–155] ns	la mediana dei modelli e, fra parentesi, il più leggero e il più pesante
85/85	vale su 85 modelli su 85
4,3 volte	un rapporto calcolato modello per modello, poi la mediana
senza rete	il nodo che riceve e inoltra senza rete neurale: il costo fisso

Il **nodo** è il computer che riceve il pacchetto, esegue la rete neurale per scegliere da quale porta farlo uscire, e lo inoltra. Il processore ha tre tipi di core: **P-core** veloci, **E-core** efficienti, **LP E-core** a basso consumo.

Il **costo** di un pacchetto è il tempo di processore che serve per elaborarlo; 1 diviso quel tempo è la **capacità**, quanti pacchetti al secondo il nodo regge.

Portatile di laboratorio (Intel Core Ultra 7 155H), core a frequenza fissa 3

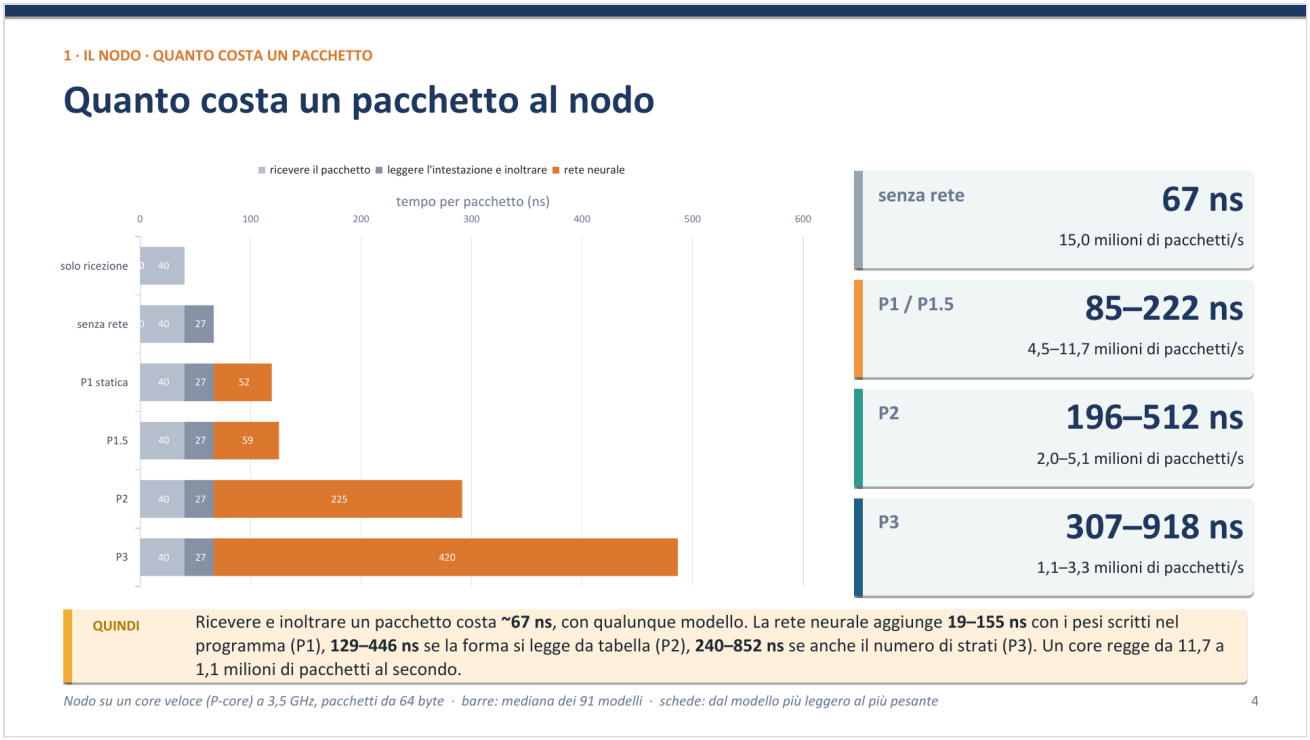
Questa slide spiega come leggere i numeri del resto della presentazione.

- **«52 [19–155] ns»**: 52 è la **mediana** sui modelli (metà dei modelli costa meno, metà di più); fra parentesi quadre il valore del modello più leggero e quello del più pesante.
- **«85/85»**: un'affermazione vale su 85 modelli su 85. Se un modello la smentisse, il conteggio lo direbbe.
- **«4,3 volte»**: un rapporto calcolato **modello per modello** (per esempio il costo di P2 diviso quello di P1 sullo stesso modello), poi riassunto con la mediana.
- **«senza rete»**: il nodo che riceve e inoltra il pacchetto senza eseguire nessuna rete neurale. È il costo fisso che ogni versione paga; il costo della **rete neurale** è quello che ogni versione aggiunge sopra.

Due grandezze tornano ovunque. Il **costo** di un pacchetto è il tempo di processore che serve per elaborarlo, in nanosecondi. La **capacità** è quanti pacchetti al secondo il nodo regge: 1 diviso il costo. Un nodo che spende 250 ns per pacchetto ne regge 4 milioni al secondo.

SLIDE 4

Quanto costa un pacchetto al nodo



Come si misura. Il generatore manda al nodo più pacchetti di quanti ne possa elaborare. La coda davanti al nodo resta sempre piena, il core lavora il 100 % del tempo, e il ritmo a cui elabora è la sua capacità.

Le barre scompongono il costo di un pacchetto in tre pezzi (mediane sui 91 modelli):

- **ricevere il pacchetto** (grigio chiaro), circa 40 ns: lo misura un programma che riceve e basta;
- **leggere l'intestazione e inoltrare** (grigio scuro), circa 27 ns: quello che il nodo senza rete fa in più;
- **rete neurale** (arancione): quello che ogni versione aggiunge.

Le schede a destra danno, per ogni versione, il costo e la capacità dal modello più leggero al più pesante.

> **Esempio di calcolo.** P2, nella mediana dei modelli, regge 3,4 milioni di pacchetti al secondo: spende 1 / 3 400 000 di secondo, circa 291 ns, per pacchetto. Il nodo senza rete ne spende 67. La differenza, circa 225 ns, è il tempo della rete neurale.

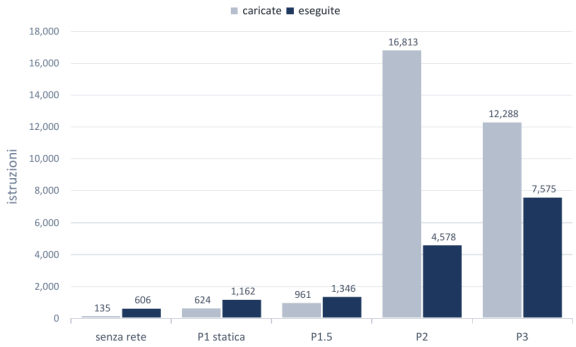
La rete neurale costa da 19 a 155 ns se i pesi sono scritti nel programma (P1), da 129 a 446 se la forma si legge da tabella (P2), da 240 a 852 se anche il numero di strati (P3). Un core regge da 11,7 a 1,1 milioni di pacchetti al secondo. L'ordine P1 ≤ P2 ≤ P3 vale su 85 modelli su 85.

SLIDE 5

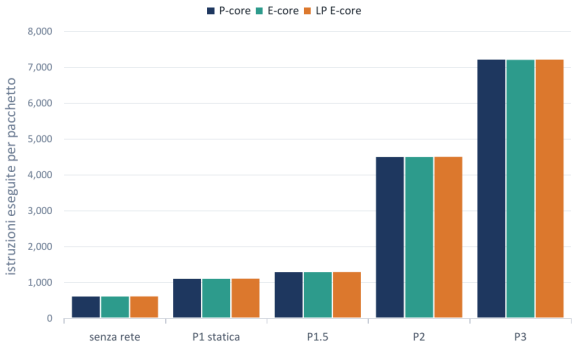
Quanto è grande il programma non dice quanto lavora

1 · IL NODO · LE ISTRUZIONI

Quanto è grande il programma non dice quanto lavora



Caricate contro eseguite. P2 è il più grande ma esegue meno istruzioni di P3 (85 modelli su 85).



Stesse istruzioni su ogni core: al più 0,3 % di differenza (0,7 % sul LP E-core).

QUINDI

La dimensione dice quanto è **grande** il programma, non quanto **lavora**: conta quante istruzioni il processore esegue davvero per ogni pacchetto. E sono le stesse su ogni tipo di core: cambia solo quante ne esegue a ogni ciclo di clock.

Caricate: la dimensione del programma, contata dal kernel · eseguite: contate dal processore per ogni pacchetto · a destra i 14 modelli misurati su tutti e tre i core

5

Grafico a sinistra. Due modi di contare le istruzioni:

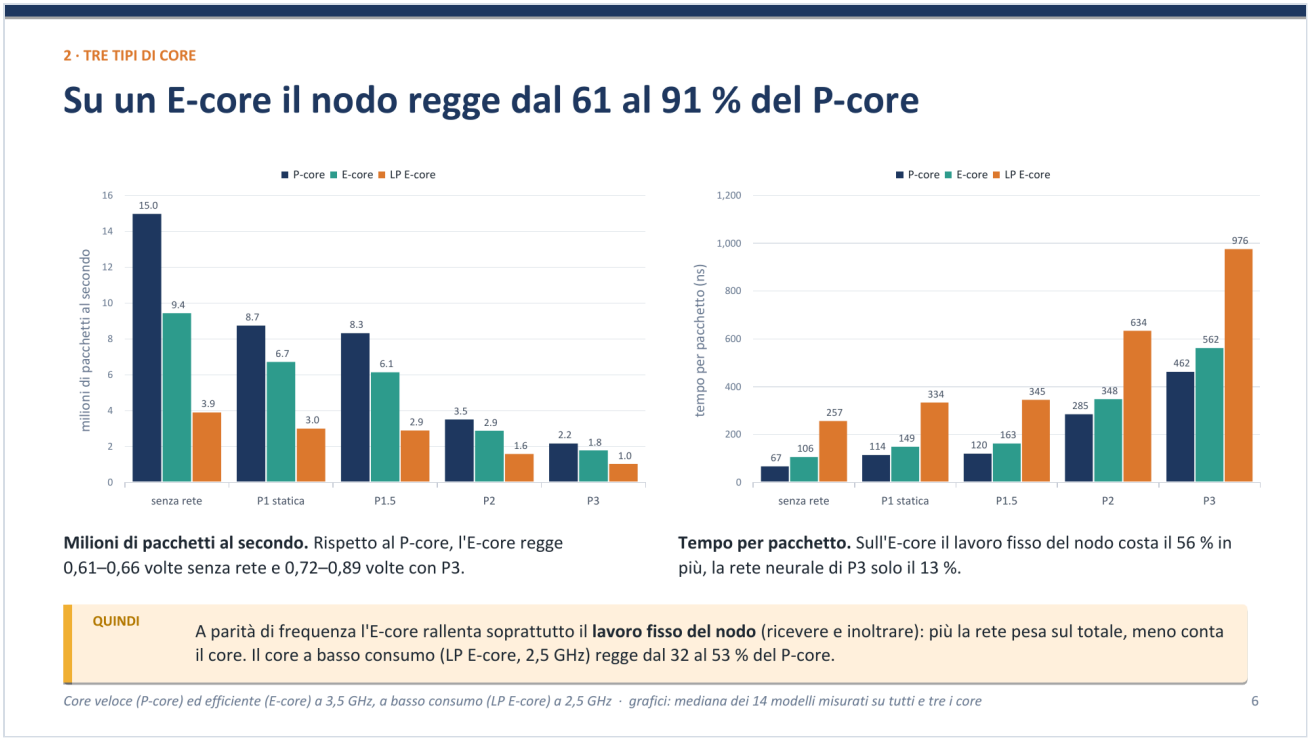
- **caricate** (grigio): quante istruzioni ha il programma. È la sua dimensione, contata dal kernel quando lo carica;
- **eseguite** (blu): quante istruzioni il processore esegue davvero per ogni pacchetto, contate dal processore stesso. Comprendono anche il lavoro del kernel per ricevere e inoltrare.

Le due misure dicono cose diverse. **P2 è il programma più grande** (circa 16 800 istruzioni) **ma ne esegue meno di P3** (circa 4 600 contro 7 600 per pacchetto), su 85 modelli su 85. P2 contiene il codice per tutte le forme ammesse, ma per ogni pacchetto ne percorre solo una parte; P3 è più piccolo (12 288 istruzioni, sempre uguale) ma ripete il suo codice una volta per strato.

Grafico a destra. Le istruzioni eseguite per pacchetto sui tre tipi di core: le barre coincidono. Il codice è lo stesso, quindi lo è anche il lavoro: fra un core e l'altro la differenza è al più dello 0,7 %. Se i core vanno a velocità diverse (slide 6), non è perché eseguono istruzioni diverse.

SLIDE 6

Su un E-core il nodo regge dal 61 al 91 % del P-core



Il processore ha tre tipi di core: sei **P-core** veloci, otto **E-core** efficienti (a gruppi di quattro che condividono una cache) e due **LP E-core** a basso consumo, in un'altra parte del chip. La domanda: che cosa succede se il nodo gira su un core più lento?

Il disegno della misura. Cambia solo il core del nodo; generatore e nodo successivo restano su P-core. P-core ed E-core girano alla stessa frequenza, 3,5 GHz, quindi la differenza dipende solo da come il core esegue il codice. Il LP E-core gira al suo massimo, 2,5 GHz.

A sinistra la capacità, **a destra** il tempo per pacchetto, nella mediana dei 14 modelli misurati su tutti e tre i core.

Sull'E-core il nodo regge circa il 63 % del P-core senza rete, e fino a circa l'83 % con P2 e P3. Più la versione è pesante, meno pesa il tipo di core. Il LP E-core regge dal 32 al 53 % del P-core.

Perché. Dividendo i cicli per pacchetto, sull'E-core la **parte fissa del nodo** (ricevere, leggere l'intestazione, inoltrare, liberare la memoria) passa da 231 a 360 cicli, il 56 % in più, su ogni modello; la **rete neurale** cambia poco: il 10–13 % in più per P2 e P3, circa lo stesso per P1. La parte fissa è codice del kernel con molti salti e accessi alla memoria, dove un core più semplice si ferma spesso ad aspettare; la rete, soprattutto in P2 e P3, è un ciclo breve e prevedibile che anche l'E-core esegue bene. Più tempo il nodo passa nella rete, meno conta che il core sia un E-core.

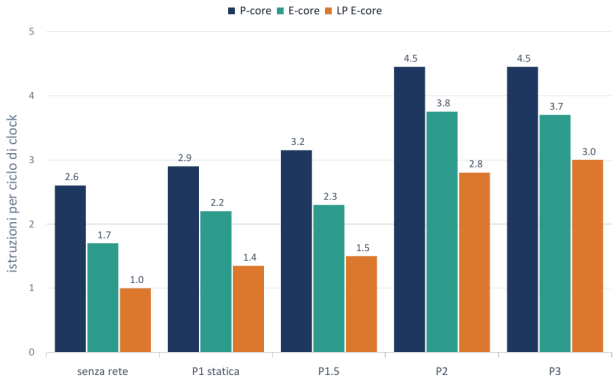
Esempio. P3, mediana dei 14 modelli: 2,2 milioni di pacchetti al secondo sul P-core, 1,8 sull'E-core, 1,0 sul LP E-core. In tempo: 462, 562 e 976 ns per pacchetto.

SLIDE 7

L'E-core esegue le stesse istruzioni, meno per ciclo

2 · TRE TIPI DI CORE · ISTRUZIONI PER CICLO

L'E-core esegue le stesse istruzioni, meno per ciclo



IPC = istruzioni ÷ cicli. Più alto non vuol dire più veloce: P2 e P3 ripetono un ciclo breve sui pesi, fatto di istruzioni semplici che il core esegue molte insieme, ma ne eseguono 4 e 6,5 volte P1.

Versione	Istruzioni	Cicli P	Cicli E
senza rete	606	231	360
P1 statica	1 162	414	531
P1.5	1 346	438	569
P2	4 578	1 016	1 218
P3	7 575	1 697	2 021

QUINDI

Fra i core cambia solo l'IPC: 2,6 → 1,7 senza rete, 4,5 → 3,7 con P3. L'E-core perde di più nel lavoro fisso del nodo, pieno di salti e attese della memoria.

Fra le versioni pesa quel lavoro fisso: in P1 è il 52 % delle istruzioni e tiene l'IPC vicino a 2,6; in P2 e P3 è il 13 e l'8 %, e conta il ciclo sui pesi.

Contatori del processore sul core del nodo · grafico: 14 modelli misurati sui tre core · tabella: mediane su 91 modelli 7

Se le istruzioni per pacchetto sono le stesse su ogni core (slide 5), la differenza sta nei **cicli di clock**: quanti battiti dell'orologio del core servono per eseguirle. Il rapporto fra istruzioni e cicli si chiama **IPC**, istruzioni per ciclo: più è alto, più istruzioni il core porta a termine a ogni battito.

Il grafico mostra l'IPC sui tre core, nei 14 modelli misurati su tutti e tre. La tabella dà istruzioni e cicli per pacchetto sul P-core e sull'E-core, mediane dei 91 modelli. La prima riga è il **nodo senza rete**: ricevere e inoltrare, il lavoro fisso che ogni versione paga.

> **Esempio.** P3 esegue circa 7 600 istruzioni per pacchetto. Il P-core ci mette circa 1 700 cicli (IPC 4,5), l'E-core circa 2 000 (IPC 3,7). A 3,5 GHz (3,5 miliardi di cicli al secondo) sono circa 485 e 577 ns: gli stessi tempi misurati con il primo metodo.

Fra i core cambia solo l'IPC. Il codice è lo stesso, e lo sono le istruzioni; l'E-core ne porta a termine meno a ogni ciclo. Il distacco è più grande dove l'IPC è già basso: 2,6 → 1,7 senza rete, 4,5 → 3,7 con P3. Il lavoro fisso del nodo è pieno di salti e di attese della memoria, e lì l'E-core, più semplice, perde di più (slide 6).

Fra le versioni l'IPC cresce, per due ragioni.

1. **Il peso del lavoro fisso.** Il nodo senza rete esegue circa 600 istruzioni per pacchetto a IPC 2,6. In P1 sono il 52 % di tutte le istruzioni, e tengono l'IPC di P1 vicino a quel valore (2,9); in P2 e P3 sono il 13 e l'8 %, e l'IPC è quasi tutto quello della rete.
2. **Come è scritta la rete.** P2 e P3 ripetono un ciclo breve su ogni peso: calcolare dove sta il peso, leggerlo, leggere l'ingresso, moltiplicare, sommare. Sono istruzioni semplici, in gran parte indipendenti da un giro all'altro, e il core ne esegue 4–5 insieme. In P1 i pesi sono scritti nel programma: il compilatore toglie indici e letture, e restano soprattutto le somme di ogni neurone, che dipendono l'una dall'altra e vanno eseguite in

fila. Contando la sola rete (tolto il nodo senza rete), l'IPC è circa 3,4 per P1, 3,8 per P1.5, 5,0 per P2 e 4,8 per P3; per P1 sale con il modello, da 2,5 sui più piccoli a 3,9 sui più grandi.

IPC alto non vuol dire veloce. P2 ha IPC 4,5 e P1 2,9, ma P2 esegue quasi 4 volte le istruzioni di P1 (4 578 contro 1 162) e spende comunque 2,5 volte i cicli (1 016 contro 414). Molte delle sue istruzioni sono il contorno (indici, letture) che P1 non ha: si eseguono facilmente in parallelo, ma vanno eseguite. Il tempo per pacchetto lo danno i cicli, non l'IPC.

Un'ipotesi, non una misura. Che P1 aspetti anche il caricamento delle sue istruzioni (il codice è tutto in fila e ogni istruzione si esegue una volta sola per pacchetto) è plausibile ma non è misurato: servirebbero i contatori che separano le attese del front-end da quelle dei calcoli.

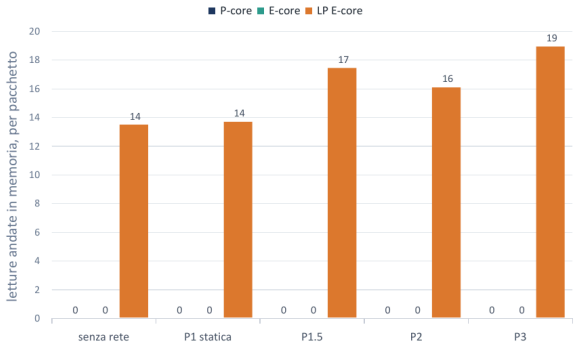
Il LP E-core ha l'IPC più basso (da 1,0 a 3,0) perché, oltre a essere più semplice, aspetta spesso la memoria (slide 8).

SLIDE 8

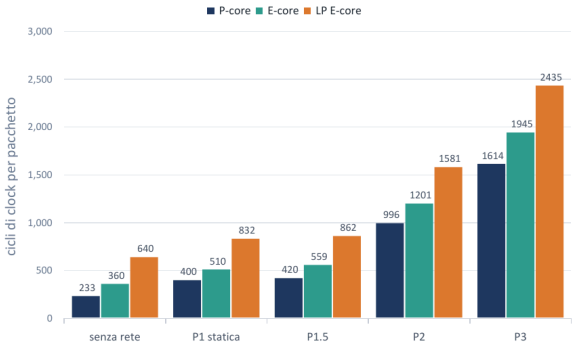
Il LP E-core paga anche la memoria

2 · TRE TIPI DI CORE · IL CORE A BASSO CONSUMO

Il LP E-core paga anche la memoria



Letture andate in memoria (il dato non è in cache): zero su P-core ed E-core, 14–19 per pacchetto sul LP E-core.



Cicli per pacchetto. Per P3: 1 614 sul P-core, 1 944 sull'E-core, 2 435 sul LP E-core; in tempo 462, 562 e 976 ns.

QUINDI

Il LP E-core sta in un'altra parte del chip, fuori dalla cache che gli altri core condividono: i pacchetti gli arrivano dalla memoria. Fra attese, un core più semplice e 2,5 GHz regge dal 32 al 53 % del P-core.

Contatori del processore, per pacchetto, a nodo pieno · LP E-core a 2,5 GHz · mediane dei 14 modelli misurati sui tre core

8

A sinistra le **letture andate in memoria** per pacchetto: quante volte il core cerca un dato nella sua cache (una piccola memoria veloce vicino al core) e non lo trova, e deve andarlo a prendere nella memoria principale, molto più lenta. Su P-core ed E-core sono zero: i pacchetti che il generatore scrive restano nella cache che questi core condividono. Il LP E-core sta fuori da quella cache: ha 14–19 letture in memoria per pacchetto, e ogni volta aspetta.

A destra i cicli per pacchetto sui tre core. Per P3: circa 1 600 sul P-core, 1 900 sull'E-core, 2 400 sul LP E-core. In tempo conta anche la frequenza più bassa del LP E-core: 462, 562 e 976 ns.

> **Esempio.** Il nodo senza rete sul LP E-core spende 640 cicli per pacchetto, contro 233 sul P-core: quasi tre volte tanto, per lo stesso codice. La differenza è quasi tutta attesa della memoria.

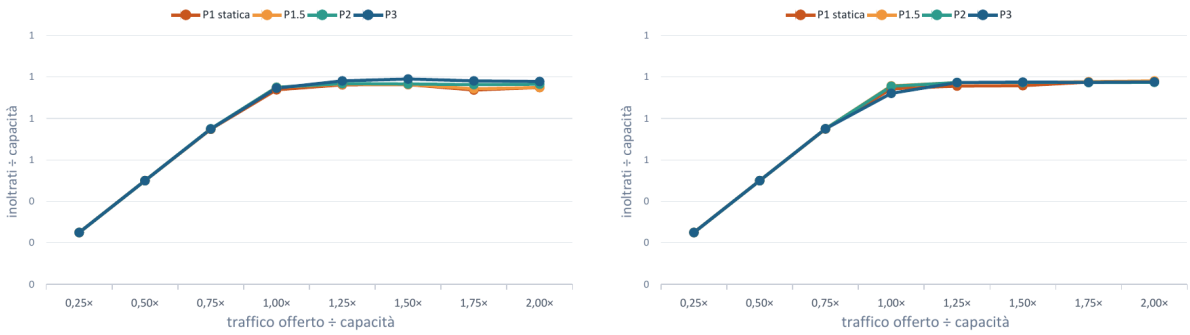
Il LP E-core regge dal 32 al 53 % del P-core. È il caso estremo: un core pensato per tenere acceso il sistema consumando poco, non per elaborare pacchetti.

SLIDE 9

Oltre la capacità si perde all'ingresso del programma

3 · SOTTO CARICO · TRAFFICO CRESCENTE

Oltre la capacità si perde all'ingresso del programma



P-core. Fino alla capacità passa tutto, poi la curva resta piatta: il massimo inoltrato è 0,98 volte la capacità misurata a parte.

E-core. Stesso andamento (0,99). Divise per la capacità del loro modello, le curve di modelli diversi coincidono.

QUINDI

Oltre la capacità i pacchetti si perdono **in coda, prima del programma**: il collo di bottiglia è il programma, su ogni tipo di core. Fa eccezione P1 sui modelli più leggeri: sopra i 9 milioni di pacchetti al secondo perde anche l'uscita.

Traffico crescente da 0,5 a 12 milioni di pacchetti al secondo, 6 modelli dal più leggero al più pesante · entrambi gli assi divisi per la capacità del modello

9

Qui il traffico **cresce** a poco a poco, da 0,5 a 12 milioni di pacchetti al secondo, e si contano i pacchetti inoltrati. A sinistra il nodo su un P-core, a destra su un E-core; sei modelli, dal più leggero al più pesante.

Modelli diversi hanno capacità diverse. Per metterli sullo stesso grafico, entrambi gli assi sono divisi per la capacità di ogni modello: sull'asse orizzontale il traffico offerto, sull'asse verticale i pacchetti inoltrati. Così «1» vuol dire «la capacità di quel modello», qualunque sia.

Ogni curva sale in diagonale (tutto ciò che arriva viene inoltrato) finché il traffico resta sotto la capacità, poi diventa piatta: il nodo non può inoltrare di più, e il resto viene respinto dalla coda d'ingresso piena. Il massimo inoltrato coincide con la capacità misurata a nodo pieno: 0,98 volte sul P-core, 0,99 sull'E-core.

Dove si perde. Il banco conta i pacchetti in tre punti: inviati, ricevuti dal programma, inoltrati. Quasi tutti i pacchetti persi si perdono **prima** del programma, nella coda: il programma inoltra tutto ciò che riceve. Il collo di bottiglia è il programma, non la trasmissione, su ogni tipo di core.

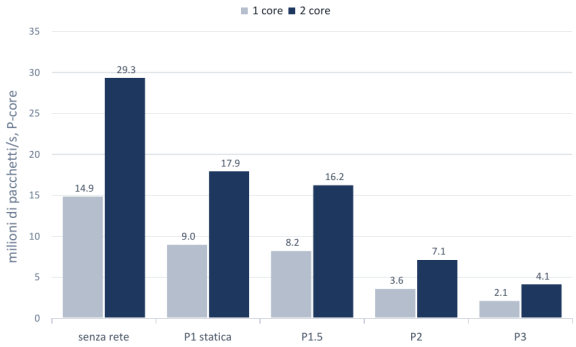
L'eccezione. P1 sui modelli più leggeri regge più di 9 milioni di pacchetti al secondo, e lì comincia a perdere anche all'uscita: è il limite del cavo virtuale e del core d'uscita, non della rete neurale.

SLIDE 10

Due core reggono il doppio, sul P-core e sull'E-core

3 · SOTTO CARICO · DUE CORE

Due core reggono il doppio, sul P-core e sull'E-core



P-core. Con due core il nodo regge il 99 % del doppio (fra il 94–103 %).



E-core. Il 99 % del doppio, con i due core in gruppi diversi del chip.

QUINDI

Ogni core ha i suoi contatori e la sua coda d'uscita: il secondo core non rallenta il primo. Una scheda di rete che divide i pacchetti su **N code** regge circa **N volte** la capacità di un core.

Una coda, un generatore e un core d'uscita per ogni core del nodo · mediane di 8 modelli dal più leggero al più pesante

10

Qui il nodo usa **due core**, ciascuno con la sua coda d'ingresso, il suo generatore, la sua coda d'uscita e il suo core d'uscita. A sinistra due P-core, a destra due E-core (in due gruppi diversi del chip). Barre chiare: un core; barre scure: due. Mediane di 8 modelli dal più leggero al più pesante.

Come si legge. P2 con un P-core regge 3,6 milioni di pacchetti al secondo, con due 7,1: il doppio. Sull'E-core 2,9 e 5,8.

Due core reggono il 99 % del doppio di un core, sia sul P-core sia sull'E-core, con ogni modello. Il secondo core non rallenta il primo perché non scrivono mai negli stessi dati: ogni core ha i suoi contatori e la sua coda d'uscita.

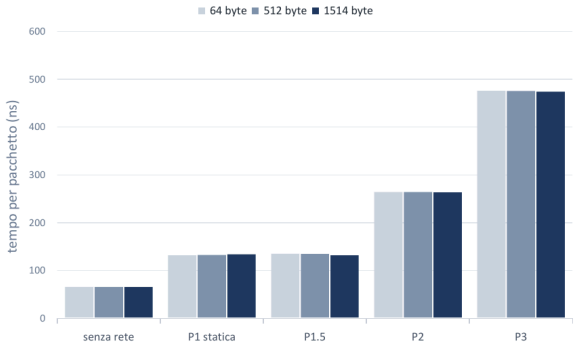
Conseguenza pratica: una scheda di rete vera divide i pacchetti su più code, ognuna servita da un core. Un nodo con N code regge circa N volte la capacità di un core.

SLIDE 11

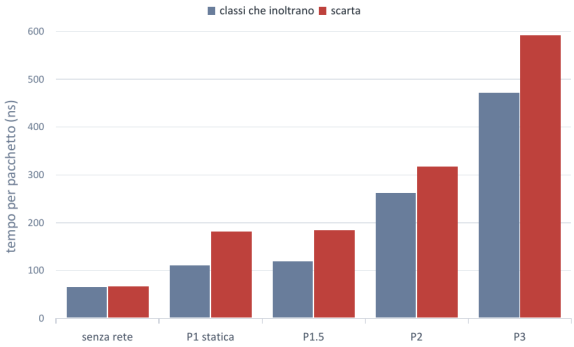
Il costo dipende solo da versione e modello

3 · SOTTO CARICO · CHE COSA NON CONTA

Il costo dipende solo da versione e modello



Taglia del pacchetto: nessun effetto (1,2 % fra 64 e 1 514 byte, in mediana). Il nodo legge solo l'intestazione.



Porta d'uscita: nessun effetto. Scartare il pacchetto costa ~51 ns in più: la sua memoria si libera sul core del nodo.

QUINDI

Basta un numero per versione e modello a descrivere il costo del nodo: non contano la taglia del pacchetto, la porta d'uscita né il TTL. Solo la decisione di scartare costa ~51 ns in più, per come è fatto il banco.

Una prova per ogni decisione che il modello sa prendere · P-core a 3,5 GHz · mediane di 6 modelli (taglie) e 8 (decisioni) 11

Tre controlli sul P-core, ciascuno su modelli dal più leggero al più pesante:

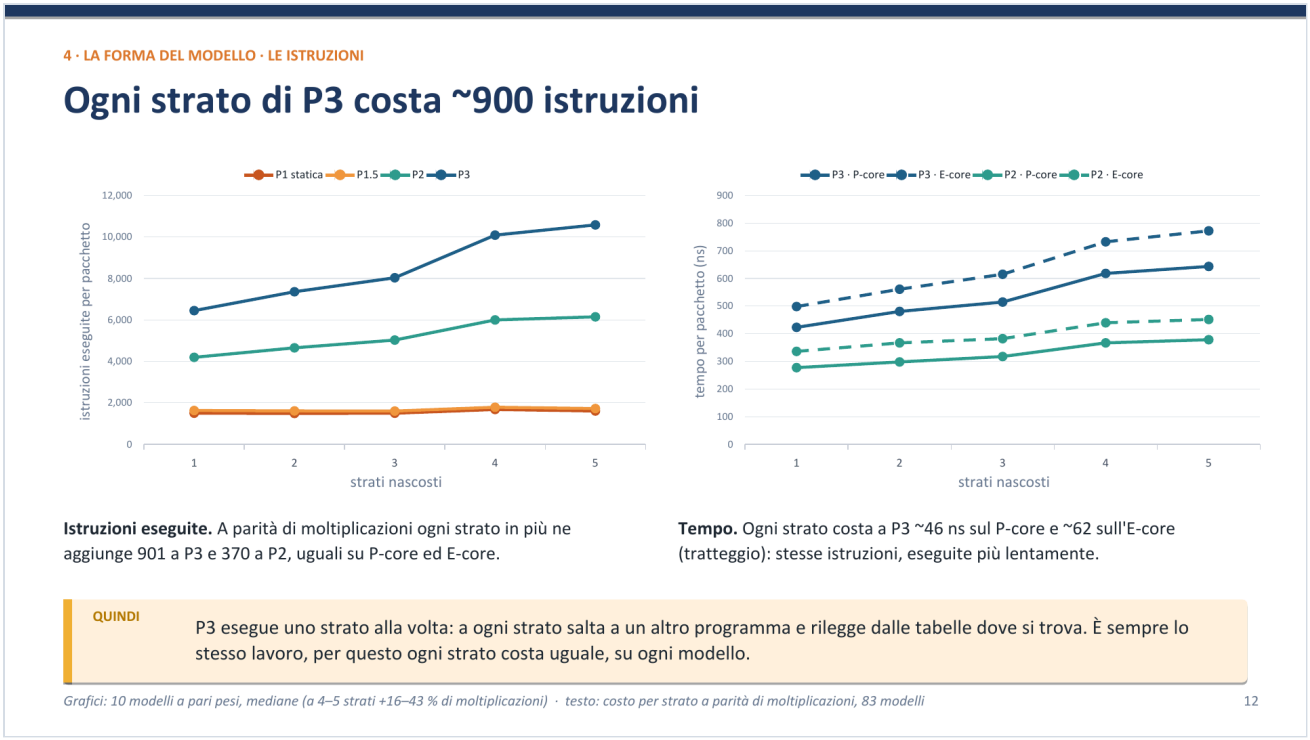
- **a sinistra la taglia del pacchetto** (6 modelli): a 64, 512 e 1 514 byte ogni versione costa uguale, con l'1,2 % di differenza in mediana. Il nodo legge solo l'intestazione del pacchetto, come farebbe su una scheda di rete vera;
- **a destra la decisione presa** (8 modelli): lo stato dei collegamenti e il TTL sono scelti apposta perché la rete decida ciascuna delle uscite che il modello sa raggiungere. Inoltrare su una porta o su un'altra costa uguale; **scartare** il pacchetto (rosso) costa circa 51 ns in più;
- nel riquadro, **pacchetti tutti diversi** (12 modelli): un TTL diverso a ogni pacchetto fa prendere al modello decisioni diverse pacchetto per pacchetto, e il costo non cambia.

Perché scartare costa di più. Quando il pacchetto è inoltrato, la sua memoria la libera il core d'uscita; quando è scartato, la libera il core del nodo, che quindi lavora di più. È un effetto del banco, non della rete neurale.

Basta quindi un numero per versione e modello a descrivere il costo del nodo.

SLIDE 12

Ogni strato di P3 costa ~900 istruzioni



I grafici usano le **forme a pari pesi**: per ognuno dei due tipi d'ingresso, cinque reti con gli stessi pesi distribuiti su 1, 2, 3, 4 o 5 strati nascosti (589–604 pesi con l'ingresso da 65 valori, 207–268 con quello da 17). Ogni punto è la mediana dei due ingressi. Lo stesso gruppo di forme torna nella slide 17.

> **A pari pesi, non sempre a pari moltiplicazioni.** Da 1 a 3 strati anche le moltiplicazioni eseguite restano uguali, entro il 3 %. A 4 e 5 strati crescono del 16–43 %, perché gli strati intermedi sono più larghi (5 e 4 neuroni invece di 3): per questo tutte le curve, anche quelle di P1, fanno un gradino a 4 strati. Il costo di uno strato a moltiplicazioni ferme lo dà la legge della slide 18; da 1 a 3 strati la misura diretta coincide con la legge.

A sinistra le istruzioni eseguite per pacchetto, contate dal processore. Da 1 a 3 strati P3 passa da circa 6 400 a 8 000 (circa 790 per strato), P2 da 4 200 a 5 000; P1 e P1.5 restano a circa 1 500. Stimato su tutti gli 83 modelli senza pesi a zero, a parità di moltiplicazioni, ogni strato in più aggiunge **circa 900 istruzioni a P3** e circa 370 a P2, uguali su P-core ed E-core. P1 e P1.5 non pagano lo strato in sé: le loro istruzioni crescono solo con le moltiplicazioni.

> **Che cosa sono le ~900 istruzioni.** P3 esegue uno strato alla volta. Alla fine di ogni strato salta di nuovo al programma generico dello strato, che deve rileggere dalle tabelle dove si trova, che forma ha lo strato e dove stanno i suoi pesi. È sempre lo stesso lavoro, per questo ogni strato costa uguale, su ogni modello.

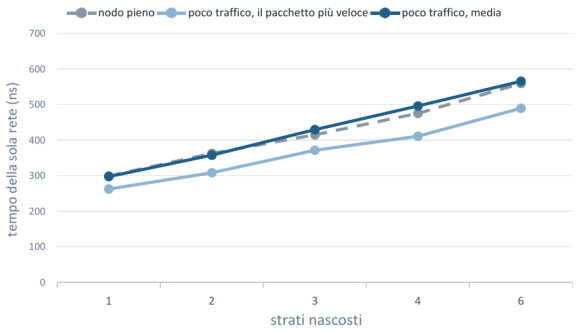
A destra il tempo per pacchetto del nodo intero, per P3 e P2, sul P-core (linea continua) e sull'E-core (tratteggio). Da 1 a 3 strati P3 sale di circa 46 ns per strato sul P-core, come stima la legge, e di circa 58 sull'E-core (la legge: 62): le stesse istruzioni, eseguite più lentamente.

SLIDE 13

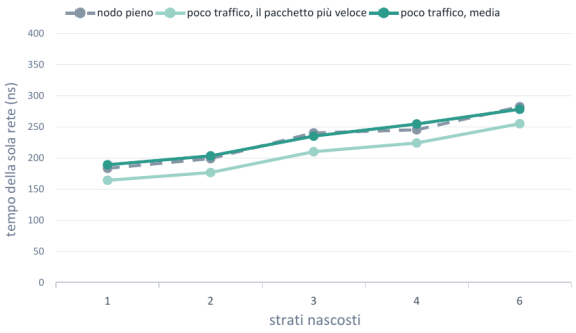
Poco traffico o nodo pieno: la rete di P2 e P3 costa uguale

4 · LA FORMA DEL MODELLO · DUE MODI DI MISURARE

Poco traffico o nodo pieno: la rete di P2 e P3 costa uguale



P3. Con poco traffico la rete costa 1,02 volte quanto a nodo pieno; il pacchetto più veloce il 14 % meno della media.



P2: 1,03 volte. Per P1 e P1.5 no (0,77 e 0,83): la loro rete costa poche decine di ns, e le due misure non la distinguono bene.

QUINDI

Due modi indipendenti di misurare la rete, a nodo pieno e con poco traffico, danno lo stesso costo per P2 e P3 su ogni modello. Il pacchetto più veloce è quello che trova la cache già pronta.

Tempo letto dal programma stesso, con 0,25 e 0,5 milioni di pacchetti al secondo, meno il nodo senza rete · mediane per strati, 53 modelli · P-core 13

Prima di passare alla sola rete neurale, un controllo su come la si misura. Il costo della rete si può ricavare in due modi indipendenti:

- **a nodo pieno**: il tempo per pacchetto meno quello del nodo senza rete (slide 4);
- **con poco traffico**: con i tre orologi (vedi «Come si misura»), il tempo del programma meno lo stesso tempo del nodo senza rete.

Ogni grafico ha tre curve, mediane per numero di strati: **nodo pieno** (grigio tratteggiato), **poco traffico, il pacchetto più veloce** (chiaro) e **poco traffico, media** (scuro). **A sinistra P3, a destra P2.**

Per P3 la media con poco traffico vale 1,02 volte il costo a nodo pieno, per P2 1,03: **le due misure danno lo stesso costo**, su ogni modello. Il pacchetto più veloce costa circa il 14 % meno della media.

Perché il pacchetto più veloce costa meno. Fra un pacchetto e l'altro il core riceve, inoltra e serve il kernel, e una parte di ciò che il programma aveva in cache viene sostituita. Il pacchetto più veloce è quello che trova tutto già pronto; la media è il costo tipico, e coincide con quello misurato a nodo pieno.

P1 e P1.5 no. Per loro le due misure non coincidono (0,77 e 0,83 volte). La loro rete costa poche decine di ns sopra un nodo senza rete di circa 67: una differenza piccola fra due misure grandi, che i due metodi stimano con errori diversi. Per P1 e P1.5 il riferimento è il tempo a nodo pieno.

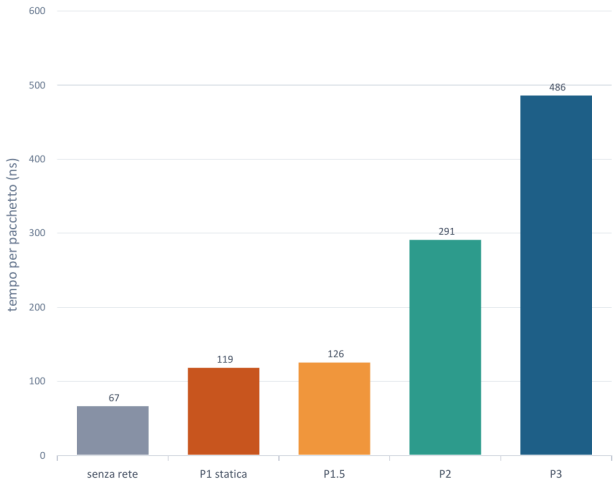
Da qui in poi ogni tempo è il **tempo a nodo pieno meno il nodo senza rete**: il costo che il nodo paga davvero per la rete neurale.

SLIDE 14

La configurabilità si paga per pacchetto: da +52 a +420 ns

5 · LA SOLA RETE NEURALE · IL PREZZO DELLA CONFIGURABILITÀ

La configurabilità si paga per pacchetto: da +52 a +420 ns



Versione	Istruzioni	Letture tabella	Salti
P1 statica	624	5	1
P1.5	961	6	1
P2	16 813	12	1
P3	12 288	23–54	2–7

QUINDI

Il prezzo della configurabilità sta nelle **letture di tabella** (da 5 a 35 per pacchetto) e nei **salti fra programmi** (P3 ne fa uno per strato), non nelle moltiplicazioni. In mediana P2 costa **4,3 volte** P1, P3 **7,8 volte**; P1.5 quasi come P1.

Mediane su 91 modelli (P2: 85) · tempo per pacchetto, P-core · letture e salti contati da una versione del programma che li conta

Il grafico dà il tempo per pacchetto in mediana sui 91 modelli: 67 ns senza rete, 119 con P1, 126 con P1.5, 291 con P2, 486 con P3. La sola rete neurale costa quindi da **+52 ns (P1) a +420 ns (P3)**.

La tabella spiega da dove viene la differenza, con tre proprietà del programma (mediane):

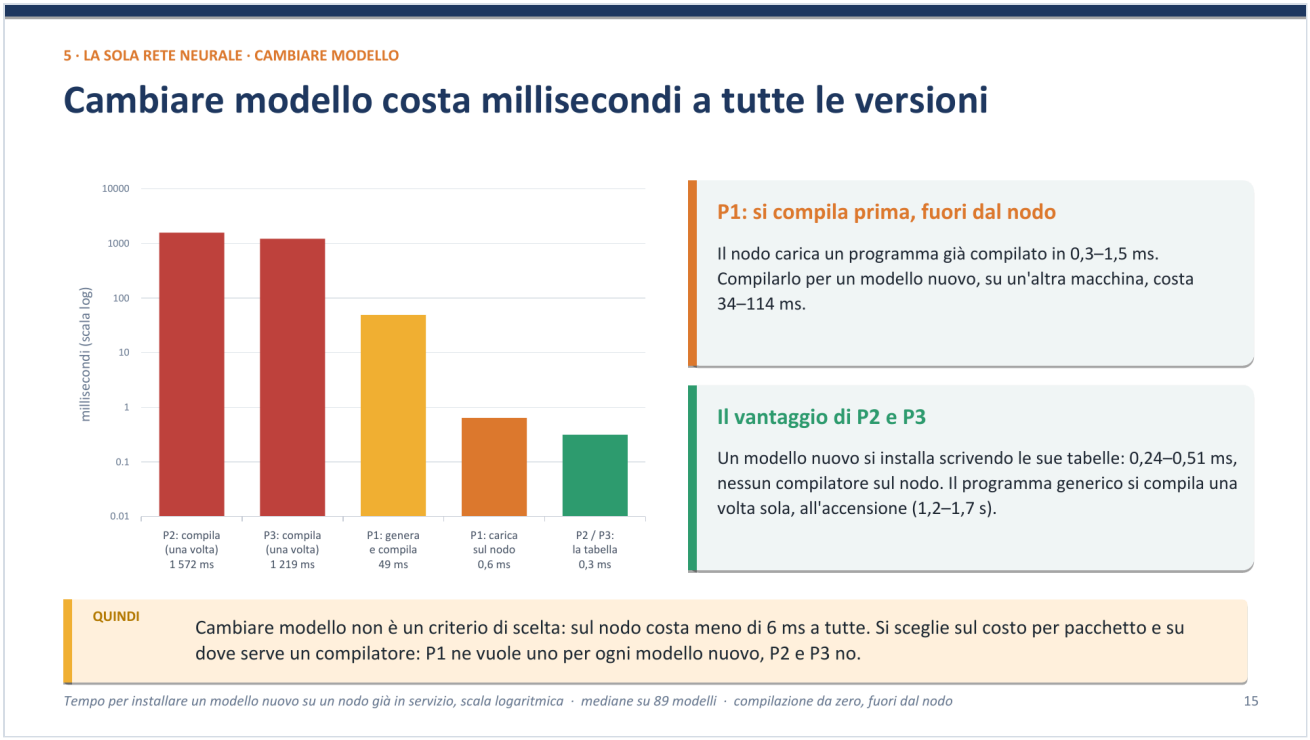
- **istruzioni**: la dimensione del programma;
- **letture di tabella** per pacchetto: quante volte il programma consulta una tabella. 5 per P1, 6 per P1.5, 12 per P2, e per P3 da 23 a 54 secondo il modello, circa 6 in più per ogni strato. Le conta una versione del programma che incrementa un contatore a ogni lettura;
- **salti fra programmi** per pacchetto: P1, P1.5 e P2 sono un programma solo, raggiunto con un salto; P3 salta una volta per ogni strato (da 2 a 7 salti), su 89 modelli su 89.

Esempio. P2 ha circa 16 800 istruzioni, P3 12 288: P3 è più piccolo, eppure costa quasi il doppio. Lo spiegano i salti (uno per strato) e le letture di tabella (35 contro 12 per pacchetto).

Il prezzo della configurabilità sta nelle letture di tabella e nei salti, non nelle moltiplicazioni. Modello per modello, P2 costa 4,3 volte P1 e P3 7,8 volte, in mediana; P1.5 quasi come P1. I rapporti più alti vengono dai modelli più piccoli, dove P1 costa poche decine di ns.

SLIDE 15

Cambiare modello costa millisecondi a tutte le versioni



Questi non sono tempi per pacchetto, ma il tempo per **installare un modello nuovo** su un nodo già in servizio. Il grafico (scala logaritmica: ogni tacca vale dieci volte la precedente) dà le mediane:

- **P2 e P3: compilare il programma generico**, una volta sola all'accensione del nodo. Ciascuna versione compila il suo: P2 circa 1,6 secondi (1,5–1,7), P3 circa 1,2 (1,20–1,23). Nel grafico sono le prime due barre;
- **P1: generare e compilare** il programma con i pesi dentro, su una macchina qualunque fuori dal nodo: 34–114 millisecondi secondo il modello;
- **P1: caricarlo sul nodo**: 0,3–1,5 ms. Il kernel controlla che il programma sia sicuro e lo traduce in codice macchina;
- **P2 e P3: scrivere le tabelle** con i pesi del modello nuovo: 0,24–0,51 ms.

Perché P1 non ha bisogno di un compilatore sul nodo. Il programma con i pesi si compila prima, su un'altra macchina (*ahead of time*), e al nodo arriva un file già pronto, che un piccolo caricatore attacca. Sul nodo non serve né il compilatore né il suo ambiente.

Cambiare modello non è quindi un criterio di scelta: sul nodo costa meno di 6 ms a tutte le versioni. Si sceglie sul costo per pacchetto (slide 14) e su dove serve un compilatore: P1 ne vuole uno per ogni modello nuovo, su qualche macchina; P2 e P3 nessuno.

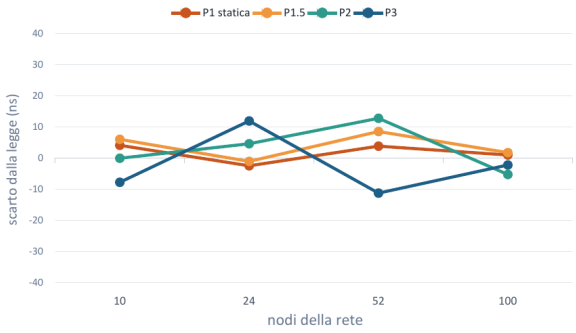
Che cosa non dice. Scrivere le tabelle è misurato su un nodo che le ha già aperte. E non dice che cosa succede ai pacchetti che passano **mentre** il modello cambia: è una prova che il banco non fa ancora.

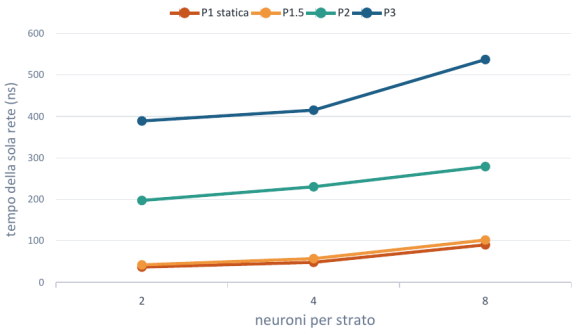
SLIDE 16

La rete può crescere quanto vuole. Il modello no.

5 · LA SOLA RETE NEURALE · TAGLIA DELLA RETE E DEL MODELLO

La rete può crescere quanto vuole. Il modello no.





Nodi della rete. Reti da 10 a 100 nodi (56 modelli): il costo resta quello previsto dalla legge, al più 0,03 ns per nodo in più.

Larghezza del modello. Da 2 a 8 neuroni sale su tutte: P1 da 37 a 90 ns, P3 da 389 a 537 ns.

QUINDI

Una **rete più grande** (più nodi) non rallenta nessuna pipeline: basta una riga in più in tabella. Un **modello più largo** rallenta tutte, a ogni pacchetto. Si può far crescere la rete; il modello va tenuto piccolo.

Tempo della sola rete, P-core · a sinistra: costo misurato meno quello previsto dalla legge, per taglia della rete · a destra: 53 modelli del disegno principale

A sinistra la taglia della rete. Fra gli ingressi di 56 modelli c'è l'identità del nodo, codificata con una colonna per ogni nodo della rete (10, 24, 52 o 100 nodi), di cui una sola vale 1. Il grafico dà, per ogni taglia, lo **scarto dalla legge**: il costo misurato meno quello previsto dalla legge della slide 18, che non tiene conto dei nodi. Se la taglia della rete costasse qualcosa, lo scarto crescerebbe con i nodi. Non cresce: oscilla di qualche ns attorno a zero, al più 0,03 ns per nodo in più.

Il motivo: delle colonne dell'identità del nodo una sola vale 1, e il programma legge solo i pesi di quella colonna, qualunque sia il numero di nodi.

A destra la larghezza del modello: più neuroni per strato. Il costo sale su tutte e quattro le versioni: P1 da 37 a 90 ns, P3 da 389 a 537 ns. Un neurone in più è lavoro vero a ogni pacchetto.

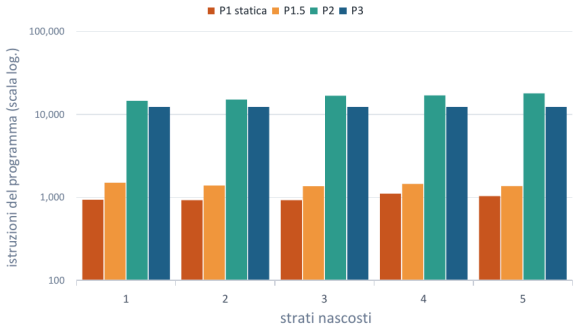
Una rete più grande non rallenta nessuna versione: basta una riga in più in tabella. Un modello più largo rallenta tutte. Si può far crescere la rete; il modello va tenuto piccolo.

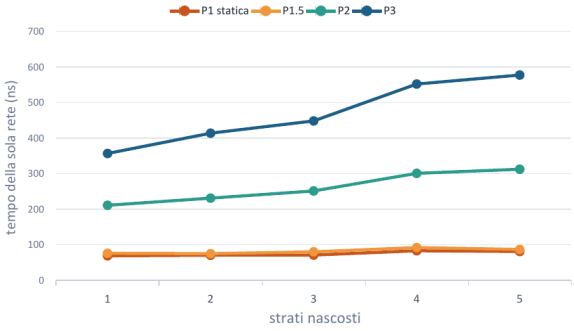
SLIDE 17

Uno strato in più: P2 lo paga in spazio, P3 in tempo

5 · LA SOLA RETE NEURALE · PROFONDITÀ

Uno strato in più: P2 lo paga in spazio, P3 in tempo





Dimensione. P2 si ricompila per ogni profondità: ~890 istruzioni in più per strato. P3 resta di 12 288 istruzioni con ogni modello.

Tempo. Da 1 a 5 strati a pari pesi: P2 +48 %, P3 +63 %, P1 e P1.5 +14–17 %. A pari moltiplicazioni: ~46 ns per strato a P3, ~20 a P2.

QUINDI

P2 paga lo strato in un programma più grande da compilare, P3 in ~46 ns a ogni pacchetto. Con P1 e P1.5 la forma della rete si sceglie per l'accuratezza; con P2 e P3, a parità di pesi, conviene una rete larga e poco profonda.

10 modelli a pari pesi, mediane (a 4–5 strati +16–43 % di moltiplicazioni) · dimensione: programma caricato · tempo: sola rete, P-core 17

Le forme a pari pesi della slide 12, da 1 a 5 strati nascosti (a 4 e 5 strati con il 16–43 % di moltiplicazioni in più).

A sinistra la dimensione del programma (scala logaritmica). P2 si ricompila per ogni profondità e scrive per esteso il codice di ogni strato: circa 890 istruzioni in più per strato. P3 non si muove: 12 288 istruzioni con ogni modello, perché esegue sempre lo stesso strato generico.

A destra il tempo della sola rete. Da 1 a 3 strati, a pesi e moltiplicazioni uguali, ogni strato costa a P3 circa 46 ns, perché è un salto fra programmi e un giro di letture di tabella (le ~900 istruzioni della slide 12); a P2 circa 20 ns; a P1 e P1.5 1–2 ns. Sono gli stessi valori che la legge stima su tutti i modelli.

Un modello profondo si paga sempre: con P2 in un programma più grande da compilare, con P3 in tempo a ogni pacchetto. P3 è la versione più flessibile, ed è quella che la profondità rallenta di più.

Meglio larga o profonda? Con P1 e P1.5 la forma conta poco (69–92 ns di rete, circa 4 ns per strato): il compilatore scrive tutta la rete per esteso in una funzione sola, e la forma si sceglie per l'accuratezza. Con P2 e P3 la profondità si paga: da 1 a 5 strati la rete costa il 48 % in più con P2 e il 63 % con P3 (contro il 14–17 % di P1 e P1.5), e a parità di pesi conviene una rete larga e poco profonda.

Esempio. Gli stessi pesi costano a P3 357 ns in un solo strato e 577 in cinque (mediana dei due ingressi): la scelta della forma vale più di 200 ns per pacchetto con P3, una decina con P1.

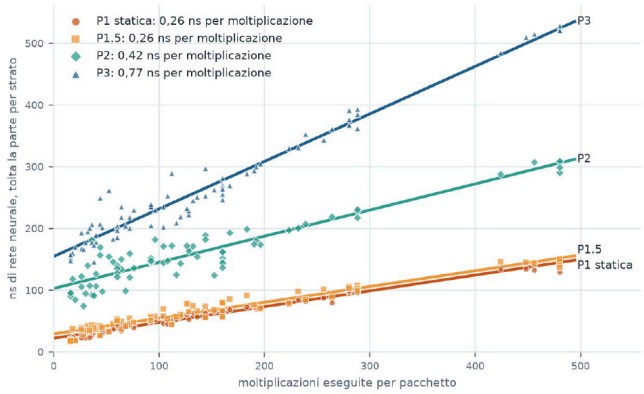
Perché da 1 a 5 strati P3 paga 58 ns per strato e non 46. A 4 e 5 strati queste forme eseguono il 16–43 % di moltiplicazioni in più, e la pendenza da 1 a 5 strati le comprende. Da 1 a 3 strati, dove le moltiplicazioni sono le stesse, uno strato costa a P3 46 ns, come stima la legge (slide 18), e la rete sale del 26 % con P3 e del 19 % con P2 invece del 63 e del 48 %.

SLIDE 18

Il costo di un modello si può prevedere prima di misurarlo

5 · LA SOLA RETE NEURALE · PREVEDERE IL COSTO

Il costo di un modello si può prevedere prima di misurarlo



Versione	fisso · moltipl. · strati
P1 statica	40 % · 50 % · 8 %
P1.5	48 % · 46 % · 5 %
P2	47 % · 18 % · 31 %
P3	37 % · 18 % · 39 %

Dove va il tempo. Con P1 e P1.5 circa metà è moltiplicazioni. Con P2 e P3 circa l'82 % è struttura: il lavoro fisso e quello per strato, cioè salti fra programmi e letture di tabella. Su modelli mai visti la legge sbaglia del 3–9 %.

QUINDI **Costo della rete ≈ fisso + moltiplicazioni × costo di una + strati × costo di uno.** P1: ~22 ns + 0,26 ns per moltiplicazione. P3: ~154 ns + 0,77 ns per moltiplicazione + ~46 ns per strato.

Stimata su 83 modelli senza pesi a zero, errore su modelli lasciati fuori · grafico: costo meno la parte per strato · tabella: quanto pesa ogni parte 18

Si può sapere quanto costerà un modello prima di misurarlo? Su tutti gli 83 modelli senza pesi a zero, il costo della rete segue una legge semplice:

costo della rete ≈ parte fissa + moltiplicazioni × costo di una + strati × costo di uno

Sul P-core:

- **P1**: 22 ns + 0,26 ns per moltiplicazione + 1 ns per strato;
- **P1.5**: 29 ns + 0,26 ns per moltiplicazione + 1 ns per strato;
- **P2**: 103 ns + 0,42 ns per moltiplicazione + 20 ns per strato;
- **P3**: 154 ns + 0,77 ns per moltiplicazione + 46 ns per strato.

Il trucco è contare le moltiplicazioni **davvero eseguite**. Un ingresso codificato con molte colonne di cui una sola vale 1 (come l'identità del nodo) occupa molte colonne di pesi, ma il programma ne legge una sola: contato dalla forma del modello sembra costoso, nel programma no.

Il grafico mostra ogni modello: sull'asse orizzontale le moltiplicazioni eseguite, su quello verticale il costo della rete tolta la parte per strato. I punti cadono sulle rette.

> **Su modelli che non ha mai visto.** La legge si stima lasciando fuori un modello alla volta e prevedendo quello: sbaglia in mediana del 3 % per P3, del 6 % per P1 e P2, del 9 % per P1.5. Il costo di un modello nuovo si stima quindi prima di misurarlo, con qualche punto percentuale di errore. (Per chi conosce la statistica: r^2 fra 0,93 e 0,99; contando le moltiplicazioni dalla forma del modello scende a 0,58–0,79.)

Dove va il tempo. La tabella dà, per ogni versione, quanto pesa in mediana ciascuna parte della legge (parte fissa · moltiplicazioni · strati):

- **P1**: 40 · 50 · 8 %; **P1.5**: 48 · 46 · 5 %. Con i pesi nel programma circa metà del tempo sono moltiplicazioni;

- **P2**: 47 · 18 · 31 %; **P3**: 37 · 18 · 39 %. Con i pesi in tabella circa l'82 % del tempo è **struttura**: la parte fissa (il primo programma, la lettura della descrizione del modello, la preparazione degli ingressi da tabella) e la parte per strato (il salto allo strato generico e la rilettura del suo contesto).

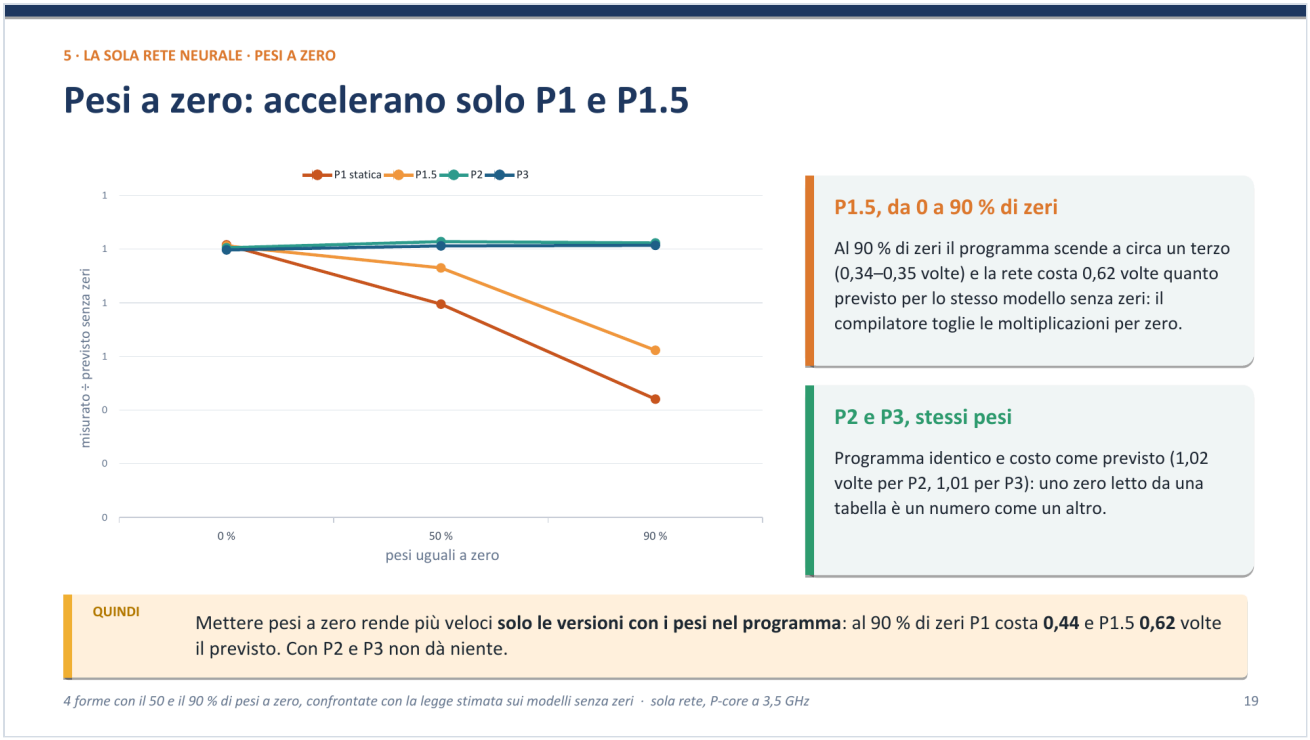
Anche il costo per moltiplicazione contiene le tabelle: P3 paga 0,77 ns per moltiplicazione contro 0,26 di P1 perché legge ogni peso da una tabella, mentre in P1 il peso è un numero scritto nel programma.

> **Nessun passaggio fuori dal kernel.** Le tabelle si leggono dentro il kernel, senza mai uscire verso i programmi normali: questi le scrivono solo quando si installa un modello. Quanto dei 46 ns per strato di P3 sia il salto e quanto le letture di tabella, la campagna non lo separa.

Anche i valori dei pesi contano poco: sei forme ripetute con pesi estratti in modo diverso danno un costo che varia al più del 5–6 % (2 % per P3).

SLIDE 19

Pesi a zero: accelerano solo P1 e P1.5



Le quattro forme con il 50 e il 90 % di pesi esattamente uguali a zero. Il grafico dà il costo misurato della rete diviso quello che la legge della slide 18 prevede per lo stesso modello senza zeri: 1 vuol dire «come previsto», sotto 1 «più veloce».

- **P1 e P1.5 accelerano**: al 90 % di zeri P1 costa 0,44 volte il previsto e P1.5 0,62. Il programma di P1.5 scende a circa un terzo delle istruzioni: il compilatore vede che un peso è zero e toglie la moltiplicazione.
- **P2 e P3 non cambiano** (1,02 e 1,01 volte il previsto), con un programma identico a quello senza zeri. Uno zero letto da una tabella è un numero come un altro, e la moltiplicazione si fa comunque.

È l'unico caso in cui dimensione del programma e tempo scendono insieme. Mettere pesi a zero (potare il modello) rende più veloci solo le versioni con i pesi nel programma; con P2 e P3 non dà niente in velocità, e va scelto solo per l'accuratezza.